

IOI 2024 Solutions: Problem Nile

Problem Information

Problem Author: Pikatan Arya Bramajati (Indonesia)

Problem Preparation: Arshia Dadras, Hazem Issa

Editorial: Agustín Santiago Gutiérrez, Pikatan Arya Bramajati

Subtasks hinting key observations

Preliminary observation

Since we have to transport all N artifacts and for each of them pay either $A[i]$ or $B[i]$, we can define a positive cost $C[i] = A[i] - B[i] > 0$ associated with transporting artifact i as a single artifact in its own boat instead of sharing a boat with another artifact. More precisely: Let S be the sum of all the $B[i]$, then if X is the set of artifacts that will be transported as a single artifact in its own boat, the total cost of transportation is $S + \sum_{i \in X} C[i]$. From now on, we will focus on the problem of pairing artifacts into boats in order to minimize the extra cost $\sum_{i \in X} C[i]$ associated with unpaired artifacts.

Subtask 1: $Q \leq 5; N \leq 2000; W[i] = 1$

In this subtask, it is always possible to put any two desired artifacts in the same boat, as $|W[p] - W[q]| = 0 \leq E[j]$ for any j .

Thus, if N is even, the extra cost is 0 as we can pair all artifacts. Otherwise, we must leave exactly one artifact unmatched, which we can choose at will and so it is optimal to choose an artifact i such that $C[i]$ is minimum.

Subtask 2: $Q \leq 5; W_i = i + 1$

For each query, notice that all that we care about is whether $E_j = 1$ or $E_j > 1$.

This is because if $E_j > 1$, we can match all artifacts if N is even, and we can leave any artifact we want as the single unmatched one if N is odd, thus giving exactly the same answer as in subtask 1. The proof is to simply match greedily the two smallest-index artifacts still unmatched that we want to match: notice that this will at most skip one artifact (the specific one we want to skip if N is odd) and so indexes of matched artifacts differ by at most 2, implying $|W[p] - W[q]| \leq 2 \leq E[j]$.

If $E_j = 1$, we can still match all artifacts for even N , but for odd N it might not be possible to freely choose any single arbitrary artifact to leave unmatched. We still want to leave exactly one artifact unmatched: the number of unmatched artifacts will be necessarily odd, and if we had at least 2 unmatched artifacts, we could choose two of them i_1, i_2 such that all artifacts in between are matched (that is, artifact k is matched for all $i_1 < k < i_2$) and change the situation by

instead matching i_1 with $i_1 + 1$, $i_1 + 2$ with $i_1 + 3$, $\dots$, $i_2 - 1$ with i_2 , thus strictly improving the solution. Note that there are $i_2 - i_1 - 1$ artifacts in between in this case, which must be an even number as otherwise those $i_2 - i_1 - 1$ artifacts would be impossible to match, since they cannot be matched with other artifacts lower than i_1 or greater than i_2 because $E_j = 1$.

Thus, when $E_j = 1$ the whole problem is to identify which artifacts it is actually possible to leave as the single unmatched element, and from those choose the artifact i with smallest $C[i]$. By exactly the same reasoning as in the previous paragraph, exactly the even values of i are possible, so we choose the smallest $C[i]$ from those.

Subtask 3: $Q \leq 5; A_i = 2; B_i = 1$

In this subtask, $C[i] = 1$ for all i . Thus we only need to minimize the number of unpaired artifacts.

Assume that we sort artifacts by $W[i]$, so that now $W[i] \leq W[i + 1]$ holds for all i . If we have $W[i + 1] - W[i] > E[j]$, then no artifact i or lower can be matched with any artifact $i + 1$ or higher, and we can split the problem into two completely independent subproblems. If we do it for all such i , we are left with subproblems inside each of which $W[i + 1] - W[i] \leq E[j]$ holds, and so by the same reasoning as in subtask 2 the minimum number of unpaired artifacts for each such subproblem is either 1 or 0, depending only on whether the number of artifacts in the subproblem is even or odd.

Full solution description

The explained solutions to the previous subtasks contain some of the key observations about the problem's structure that are required for a full solution. We will now put them all together efficiently.

First, let's sort the artifacts in increasing order of $W[i]$. We will always refer to artifact indexes with respect to this order. Next, let's denote a configuration of pairings as an array of pairs $[(L_1, R_1), (L_2, R_2), \dots, (L_k, R_k)]$ such that $L_i < R_i$ and $L_i < L_{i+1}$.

Notice that any configuration where there exists a pair of indices (i, j) such that $L_i < L_j < R_j < R_i$ can always be changed into pairing artifact L_i with L_j , and pairing artifact R_j with R_i , for the same total cost. Similarly, any configuration where there exists a pair of (i, j) such that $L_i < L_j < R_i < R_j$ can always be changed into pairing artifact L_i with L_j and pairing artifact R_i with R_j . Therefore, from now on we will just consider configurations with $R_i < L_{i+1}$. Notice that any such configuration where there exists (L_i, R_i) such that $R_i - L_i \geq 3$ is never optimal, because we could instead pair artifacts L_i with $L_i + 1$ and pair artifact $R_i - 1$ with R_i , to get a lower total cost. Therefore, we only consider configurations with $R_i - L_i \leq 2$.

To do this, we can consider all $O(N)$ pairs of artifacts $(i, i + 1)$ and $(i, i + 2)$. We sort those pairs by increasing W difference (that is, for pair (a, b) we consider the difference $W[b] - W[a]$).

Also, we sort the queries array E in non decreasing order. Then, a two-pointer algorithm can be used: we iterate the queries, while simultaneously iterating the array of sorted pairs, and at each step we add to the current set of available pairs, all extra pairs satisfying $W[b] - W[a] \leq E[j]$.

We maintain connected components of artifacts using a Disjoint Set Union data structure. Initially, each artifact is its own connected component. Note that connected components are always ranges of consecutive artifacts, so that it is always possible to match artifacts i and $i + 1$ belonging to the same component.

When adding a pair $(i, i + 1)$, we connect artifacts i and $i + 1$ into the same connected component. At any moment, it is possible and optimal to leave 0 unmatched artifacts in even-sized components, and exactly one unmatched artifact in each odd-sized component.

An artifact in an odd-sized component can be the single unmatched artifact, if its index inside the component is even, that is, if its index has the same parity as the minimum index in that connected component. Furthermore, adding a pair $(i, i + 2)$ adds artifact $i + 1$ as an additional valid candidate for the single unmatched element in its connected component. For each connected component with odd size, we just need to find the minimum $C[i]$ among all candidates.

We can maintain the current total cost, and the candidates summary information for each component, throughout the whole process to answer each query. Candidates summary information for each component consists of the component's size, the minimum index of an artifact in the component, the minimum odd indexed $C[i]$, the minimum even indexed $C[i]$, and the minimum $(i, i + 2)$ -enabled $C[i]$, all of which can be updated in constant time when merging two components or enabling a pair $(i, i + 2)$.

The time complexity of the solution is $O(N \log N + Q \log Q)$

A similar complexity can be achieved using segment tree, and using the same key ideas of decomposing into components: each component will be a range, so that the component's relevant information can be quickly computed by means of the segment tree data structure.

Subtask 4: $Q \leq 5; N \leq 2000$

Get the observation about $R_i < L_{i+1}$. Then, use dynamic programming. Define $dp[i]$ as the minimum total cost for all artifacts from 0 to i . When calculating $dp[i]$, we can try all possible ways to pair artifact i with any artifact $j (j < i)$ having $W[i] - W[j] \leq D$. This does not require the observation about $j - i \leq 2$.

Subtask 5: $Q \leq 5$

Use the same dynamic programming approach as in subtask 4, but only try pairing artifact i with $i - 1$ or $i - 2$.

Subtask 6: $A_i = 2; B_i = 1$

In this subtask, $C[i] = 1$ for all i . Thus we only need to minimize the number of unpaired artifacts.

We can use the same idea as in the full solution, but the solution for each component is much easier as it is simply 1 unmatched artifact if the component size is odd, or 0 unmatched artifacts if it is even. Thus, component size parity is the only information required from the DSU data structure.

IOI 2024 Solutions: Problem Message

Problem Information

Problem Author: Arthur Nascimento (Brazil)

Problem Preparation: Mahmoud Badawy, Bartosz Kostka, Mohamed Anany

Editorial: Masataka Yoneda, Hirotaka Yoneda

1. Subtask 1 (10 points)

In this subtask, consider sending one bit of message per packet. We can use the following strategy:

- **Aisha's Strategy:** If she wants to send 0, she sends a packet with 000...000. If she wants to send 1, she sends a packet with 111...111.
- **Basma's Strategy:** In the packet which Basma received, if (number of 0's) > (number of 1's), she knows that the message is 0. Otherwise, she knows that the message is 1.

This strategy works because, Cleopatra can only change at most 15 bits, which is less than half of the packet, so the correct bit remains the majority. In coding theory, this method is called **repetition code**.

2. Subtask 2: $Q = 96$ (29.31 points)

Basic Insights

First, consider for simplicity that the message length S always equals 1024.

To get some insights of the solution, thinking about some easier variation of the problem may work. This time, let's think about the following question: "*what if Basma knew precisely which 15 indices are controlled by Cleopatra?*" (we will call them 'controlled bits' from now on). In this hypothetical scenario, Aisha can send 16 bits of message per packet, by simply embedding the message in 16 non-controlled bits. Therefore, it only requires $Q = \lceil \frac{1024}{16} \rceil = 64$ packets. (Note: $\lceil x \rceil$ is the smallest integer which is at least x)

The problem is that Basma does not actually know the 15 non-controlled bits. One way to tackle the problem is to think about the strategy to convey the positions of 15 non-controlled bits, by using some packets.

The Solution when $S = 1024$

Aisha uses first 31 packets to tell which bits are non-controlled. Using the idea of Subtask 1, in $(i + 1)$ -th packet ($0 \leq i \leq 30$), Aisha sends 000...000 when $C_i = 0$, and Aisha sends 111...111 when $C_i = 1$. After completing this process, now Basma knows which bits are controlled. Then, Aisha can send the entire

message in the next 64 packets using the strategy explained in **Basic Insights** section. Overall, it requires $Q = 31 + 64 = 95$ packets.

The Solution for General S

The actual problem is that $1 \leq S \leq 1024$ and Basma does not know the message length S beforehand. However, the action of sending a message of length between 1 and 1024, can be transformed to the action of sending a message of length **exactly 1025**. This is because, the number of species of messages is $2^1 + 2^2 + \dots + 2^{1024}$, which is slightly less than 2^{1025} .

One way of implementing this is to first send 0's before the message, then a 1 to denote the beginning of the message, and finally the actual message, so that the total length is exactly 1025. For example, if the message is 101...100 with length 1019, we send the following string (underlined section is the actual message):

000001101...100

Back to the main problem, it requires $Q = 31 + \lceil \frac{1025}{16} \rceil = 96$ packets.

3. Subtask 2: $Q = 85$ (43.97 points)

The main focus on the next step is: “*how to convey which bits are controlled, in less than 31 packets?*”.

The problem is that we only sent 1 bit of information per packet, which may be inefficient. For example, one can notice that, if Basma *already* found 2 non-controlled bits, Aisha can send 2 bits of information by embedding it in these bits, which is more efficient. (Note: It can be compared to the latter phase of $Q = 96$ solution, when Basma already found 16 non-controlled bits and Aisha sends 16 bits of information.)

We consider the situation that Aisha already sent the information of $C_0, C_1, \dots, C_{i-1}$, and k of them are 0 (non-controlled bits). Then instead of sending 000...000 or 111...111, Aisha sends the next k values of the C_i 's.

The worst case is that the first 15 bits are controlled bits ($C_0 = \dots = C_{14} = 1$) and the last 16 bits are non-controlled bits ($C_{15} = \dots = C_{30} = 0$). Even in this case, the following thing happens:

- Aisha sends the information of $C_0, \dots, C_{16}$ in the first 17 packets.
- Aisha sends the information of C_{17}, C_{18} in the next packet.
- Aisha sends the information of $C_{19}, C_{20}, C_{21}, C_{22}$ in the next packet.
- Aisha sends the information of $C_{23}, \dots, C_{30}$ in the next packet.

So the phase of conveying which bits are controlled is done in 20 packets. Back to the main problem, it requires $20 + 65 = 85$ queries.

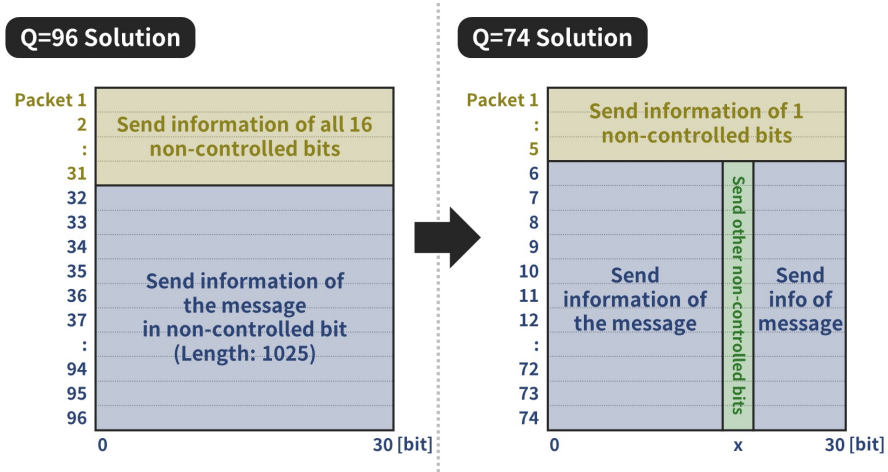
4. Subtask 2: $Q = 71$ (79.64 points)

For the solutions in Chapters 2 & 3, Aisha sends the information of all 16 non-controlled bits first and then sends the information of the message.

However, with the idea of “sending only 1 non-controlled bit first”, we can achieve $Q = 74$ by the following algorithm.

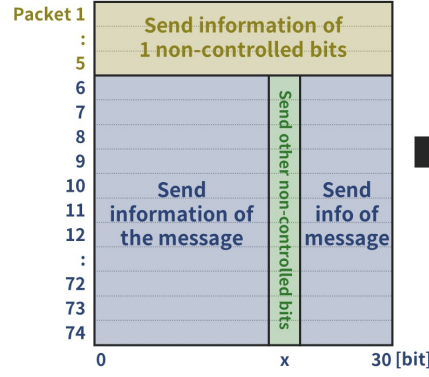
1. For the first 5 packets, Aisha sends the index of 1 non-controlled bit x , with the idea of Subtask 1.
2. For the next 69 packets, Aisha sends the index of remaining 15 non-controlled bits **in x -th bit in the packet**, and embed the content of message in the other non-controlled bits.

The idea of the algorithm is described in the figure below. Note that since we can send $69 \times 15 = 1035$ bits for the information of the message, which is greater than 1025 bits, the number of packets is enough.

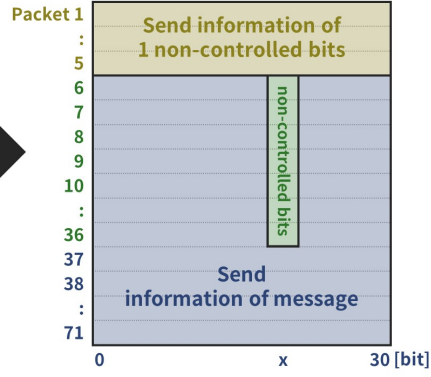


We can make a small improvement: since 31 bits is enough to send the information of the other 15 non-controlled bits, we can send the information of M even in the x -th bit after $(31 + 5 =) 36$ -th packets (see figure below). With this idea, we can achieve $Q = 71$. It suffices to get around 80 points.

Q=74 Solution



Q=71 Solution



5. Subtask 2: $Q = 66$ (100 points)

The idea for the full solution is completely different from Chapter 4. First, let's think about sending all information of non-controlled bits in 5 packets.

The idea is to send "the index of the next non-controlled bit". Let $a_1, a_2, \dots, a_{16}$ be the index of non-controlled bits in increasing order (that is, $a_i < a_{i+1}$). If Aisha sends the binary representation of a_{i+1} in the a_i -th bit of the packet (the exception is $i = 16$, in this case " a_{17} " is the alias of a_1), Basma can know the values of $a_1, a_2, \dots, a_{16}$ regardless of Cleopatra's action.

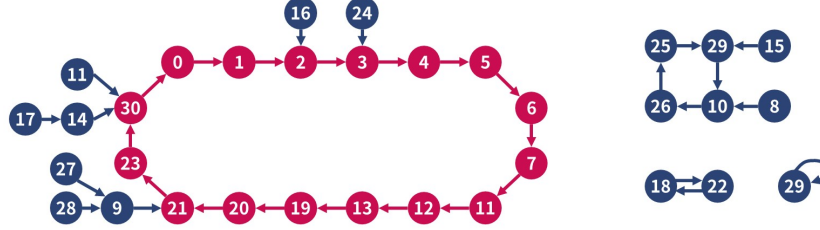
For example, if $a = [0, 1, 2, 3, 4, 5, 6, 7, 11, 12, 13, 19, 20, 21, 23, 30]$, the first 5 packets should be like the following figure. In the $(a_1 = 0)$ -th bit, since the binary representation of $a_2 = 1$ is 00001, Aisha should send 1, 0, 0, 0, 0 in the 1, 2, 3, 4, 5-th packets, respectively. Note that for the controlled bits like the 8-th bit, anything is OK because Cleopatra can change freely.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Non-controlled	●	●	●	●	●	●	●					●	●	●						●	●	●	●								●
Packet 1	1	0	1	0	1	0	1	1				0	1	1						0	1	1		0							0
Packet 2	0	1	1	0	0	1	1	1	(anything is ok)			0	0	1						0	0	1		1							0
Packet 3	0	0	0	1	1	1	1	0	(anything is ok)			1	1	0						1	1	1		1							0
Packet 4	0	0	0	0	0	0	0	1	(anything is ok)			1	1	0						0	0	0		1							0
Packet 5	0	0	0	0	0	0	0	0				0	0	1						1	1	1		1							0

So, how does Basma know the values of $a_1, a_2, \dots, a_{16}$? Let the binary representation of b_i be the i -th bit along the first 5 packets that Basma received (e.g. in the figure above, $b_0 = 1$), and consider about the following directed graph.

- Vertex: 31 vertices labeled from 0 to 30
- Edge: For each i ($0 \leq i \leq 30$), there is an edge $i \rightarrow b_i$

In this graph, regardless of Cleopatra's action, there is **exactly one** "cycle of length 16", and this cycle contains vertex $a_1, a_2, \dots, a_{16}$. Therefore, by finding a cycle of length 16, Basma can know all non-controlled positions. An example of a graph is depicted below (red vertices correspond to non-controlled bits).



Therefore, we can achieve $Q = 70$, because another 65 packets are needed to send the message.

Moreover, $Q = 69$ can be achieved with a small improvement: instead of sending the binary representation of a_i , think about sending $(a_{i+1} - a_i) \bmod 31$. Since we can easily prove that $1 \leq [(a_{i+1} - a_i) \bmod 31] \leq 16$, only 4 packets are needed to send the information of non-controlled bits.

The final step towards the full solution is to change the encoding method: from binary encoding to one-hot encoding. More precisely, we can encode the value as follows.

Value	1	2	3	4	5	...
Encoded	1	01	001	0001	00001	

The content of packets when $a = [0, 1, 2, 3, 4, 5, 6, 7, 11, 12, 13, 19, 20, 21, 23, 30]$ is described in the following figure. Note that the remaining non-controlled bits, that are not used in encoding of $a_{i+1} - a_i$ (blue area of the figure), will be used to send the information of S .

Since one-hot encoding requires $\sum [(a_{i+1} - a_i) \bmod 31] = 31$ bits, to get 100 points ($Q = 66$), $66 \times 16 - 31 = 1025$ bits are remaining. And, it is exactly the length of message to send. A magical solution!

	Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Non-controlled		●	●	●	●	●	●	●					●	●	●						●	●	●	●								●
Packet 1		1	1	1	1	1	1	0					1	1	0						1	1	0		0							1
Packet 2								0						0									1		0							
Packet 3								0						0											0							
Packet 4								1						0											0							
Packet 5									(anything is ok)					0											0							
Packet 6									(anything is ok)					1											0							
Packet 7																									1							
Packet 8																																
Packet 9																																
Packet 10																																
Packet 11																																
Packet 12																																

IOI 2024 Solutions: Problem Tree

Problem Information

Problem Author: Pikatan Arya Bramajati (Indonesia)

Problem Preparation: Mohammadreza Maleki, Franciszek Witt, Kian Mirjalali

Editorial: Agustín Santiago Gutiérrez, Mohammadreza Maleki, Yiping Liu

Solution description

Various lemmas will be used throughout this solution detailed explanation. As their proof is generally quite involved, they are placed in a separate section after the solution description, so that it is easier to follow the explanation of the solution ideas.

Let $S[i]$ be the sum of $C[j]$ for all vertices j that are in subtree of vertex i . Thus, an array C is valid if for every vertex i , $L \leq S[i] \leq R$.

Subtasks 1,2,3 : $Q \leq 10$

We will explain an algorithm, which we will simply call BottomUpGreedy, that allows us to solve the first subtasks. Subtasks 1 and 2 allow for simpler or inefficient implementations of BottomUpGreedy.

- (Lemma 1) There is always an optimal solution where leaves have $C[i] = L$.
- (Lemma 2) There is always an optimal solution where $C[x] = L$ if x is a leaf node and $C[x] \leq 0$ otherwise.

Due to these lemmas, we can enforce both restrictions and pretend that the original problem requires us to set $C[i] = L$ at leaves and to choose values $C[i] \leq 0$ for the non-leaf nodes, while satisfying all the restrictions with minimum cost.

BottomUpGreedy is the following algorithm:

- Initialize $C[i] = 0$ at every node.
- Set $C[i] = L$ at every leaf node i .
- Process nodes in a bottom up fashion (e.g. by decreasing id)
- At each node i , while $S[i] > R$, choose a valid node j in the subtree of i having the minimum possible value of $W[j]$, and decrement $C[j]$ by one. A node j is called valid if after decrementing $C[j]$ by one, all of the restrictions still hold, i.e. $S[k] \geq L$ still holds for all ancestors k of j after decrementing.

The lemmas directly justify the first steps of BottomUpGreedy, but a detailed proof of correctness is quite involved.

- (Lemma G) BottomUpGreedy is optimal.

Subtask 1 allows us to implement BottomUpGreedy in a very simple way because we can always choose $j = i$.

Subtask 2 allows us to implement BottomUpGreedy in quadratic time, by explicitly searching for the minimum weight valid node in the subtree.

Subtask 3 requires an efficient implementation that can quickly find the minimum weight valid node in the subtree. This can be done with standard data structures such as priority queues / sets / segment trees.

Subtasks 4,5 : $W[i] \leq 1$

The problem with the previous solution in the case in which we have many queries is that we would need to run BottomUpGreedy from scratch for each query. This is because the $C[i]$ computed during the first steps of the algorithm are needed for later steps, and all of those values depend on L and R , which change every query.

We can develop a more involved algorithm, whose execution flow does not heavily depend on the values of L and R , and thus it can be run a single time to precompute enough information to solve any $[L, R]$ query. We need some additional lemmas:

- (Lemma 3) If $W[i] = 0$ for non-leaf node i , then “cutting i from its children” does not change answer. That is, adding the solutions to the resulting trees as if they were each an independent instance of the problem gives an optimal solution to the original.
- (Lemma 4) An optimal solution exists with the form of lemma 2, and having either $C[i] = 0$ for every non-leaf node i , or else $S[0] = R$.

By Lemma 4 we can deduce that answer to subtask 4 is given by the formula $kL + \max(0, kL - R)$, where k is the number of leaves in the tree.

We can combine this knowledge with lemma 3 to have a solution for subtask 5: after we cut at nodes having $W[i] = 0$, we just have to add many instances of subtask 4. Efficiently handling this sum requires some prefix-sum techniques, similar to what is explained for the general solution.

General solution

We need a final critical lemma to be able to reach an efficient full solution:

- (Lemma 5) Let k be the number of leaves in the tree, and m be the minimum $W[i]$ among all the **non-leaf** nodes i . If $kL - R > 0$, then the total cost of an optimal solution is $(kL - R)m + s$, where s is the cost of the optimal solution to a modified input problem in which m is subtracted from $W[i]$ for every **non-leaf** i .

We can create a new TopDownGreedyRecursive algorithm.

Suppose for now that L and R are known and fixed just as before. To solve a subtree (initially we go for the whole input tree), we first check if $kL - R \leq 0$ and if so, we could just stop and return L times the sum of leaves' weights. Otherwise we find a minimum weight **non-leaf** node v in the subtree and set the answer to $(kL - R)W[v]$, in accordance with lemma 5.

Then we subtract $W[v]$ from $W[i]$ for each **non-leaf** node i in the subtree. Crucially this causes the new minimum weight to be $W[v] = 0$ and thus by lemma 3 we can now split the problem and recursively solve each subtree, adding all those results to our final answer.

The base case is that of a leaf node (and in that case the subtree has no non-leaf node to choose as minimum), for which the answer is just $W[i]L$.

For both efficiency and simplicity of implementation, it is better not to subtract to all weights explicitly, but rather when calling the subproblem recursively pass an integer indicating how much has to be subtracted from every weight in the calculations.

Also in order to be able to efficiently find the number of leaves k in the subtree and the minimum weight non-leaf node v , a segment tree with the so called “euler-tour tree-flattening” technique can be used. Note that a normal (non lazy) one-dimensional segment tree is enough, and it allows modifications which are necessary when cutting at v .

Specifically, the implementation can call the lower subtrees first (so one recursive call to each child of v) while the subtree containing the root is solved last, and so if the algorithm is made to “delete” (that is, set to neutral values) processed nodes from the segment tree, then the segment tree will contain precisely the right state for the final call and thus any “cutting” at node v is simply managed by consuming the tree in the correct order and doing single-node-deletions from the segment tree.

While all of the previous can be implemented in $O(N \lg N)$ time for a single run with known L and R , we need to see how to solve the problem with a single run of TopDownGreedyRecursive. The key to notice is that, apart from returning early from some call when $kL - R \leq 0$, the L and R values do not influence the decisions and execution of the algorithm at all: we always choose min weight node in subtree, cut it, and call recursively, a process which depends only on W and not on L, R .

So we can do a single run at the start where we keep iterating for generic L, R values, and we will end up with a sum of $(kL - R)W'[v]$ terms. Note that $W'[v]$ is the known “subtracted” weight at that call, which might differ from input $W[v]$. Also, each term has its own value of k , depending on the number of leaves in the subtree at that call.

Let s be the sum of weights of all leaves. A fixed factor of Ls must be added together with each of the previous terms to get the final answer. Note that each

$(kL - R)W'[v]$ term must be added only when the corresponding $kL - R > 0$, as otherwise the algorithm would not reach that sum.

This can be rewritten as $k > \frac{R}{L}$, and k can only take values in $[1, N]$. So our situation for a particular $[L, R]$ query is that for each term in the generic run of TopDownGreedyRecursive, we need to add either 0 or some known linear combination $aL + bR$ to the answer, depending only on the value of $\min(N, \frac{R}{L})$. Using the prefix-sums technique, we can use the single generic run of TopDownGreedyRecursive to precompute arrays a, b of size N such that the answer to a query $[L, R]$ is $a[t]L + b[t]R$ where $t = \min(N, \lfloor \frac{R}{L} \rfloor)$.

Proof of Lemmas

- (Lemma 1) There is always an optimal solution where leaves have $C[i] = L$.

Proof: to find one, take any optimal solution and take a leaf x having coefficient $L < C[x] \leq R$. By subtracting one to the coefficient in this leaf so that it becomes $C[x] - 1$, no condition $S[j] \leq R$ can fail because the $S[j]$ have not increased.

Also, condition $S[j] \geq L$ can only break at ancestors of x . If none breaks, then we have changed to a solution with lower or equal cost and decreased sum of leaf coefficients. Otherwise, if we consider the first (farthest from root, nearest to x) ancestor j where the $S[j] < L$, since that node has at least one child and all of those have sum at least $L > 0$, then it must be the case that $C[j] < 0$. Thus we can increment $C[j]$ by one and the total cost does not increase, but the $S[k]$ values from j up to the root now remain unchanged from the original solution (since we decremented $C[x]$ but incremented $C[j]$).

In both cases, we are able to reduce the sum of leaf coefficients, and so by repeating this process we get to an optimal solution with $C[i] = L$ at every leaf node i .

- (Lemma 2) There is always an optimal solution where $C[x] = L$ if x is a leaf node and $C[x] \leq 0$ otherwise.

Proof: Take a solution with $C[i] = L$ at every leaf node i , by lemma one. Now suppose that there exists some non-leaf node i such that $C[i] > 0$. Let v be such a node that is as close to the root of the tree as possible. Let w be the deepest ancestor of v (that is, furthest from the root) having $C[w] < 0$ (thus, each ancestor i of v that is strictly between v and w has $C[i] = 0$). Note that if all ancestors of v have coefficient 0, then w does not exist.

After decrementing $C[v]$ by one, all $S[j] \leq R$ conditions will still hold. Notice also that no condition $S[j] \geq L$ can fail at nodes deeper than w because of this change: the node v itself has at least one child and thus by having $C[v] > 0$, then $S[v] \geq L + 1$ before modification, and the same holds for the ancestors of v below w which have $C[i] = 0$ and thus $S[i]$ directly equal to sum of children $S[j]$.

So if w does not exist, we can simply decrement $C[v]$ and get a new optimal solution. Otherwise, if apart from decrementing $C[v]$ we increment $C[w] < 0$, we also get a new optimal solution. By repeating this process we keep reducing the sum of the positive non-leaf coefficients, until there are none and thus our proposed optimal solution exists.

- (Lemma G) BottomUpGreedy is optimal.

Proof: Due to the previous two lemmas, a correct solution exists that can be reached by doing some decrements of the $C[j]$, and thus the algorithm can only fail because it incorrectly chooses which nodes to decrement.

Suppose that BottomUpGreedy is not correct for a test case, and consider the first time that it makes a mistake, that is, while processing some node i , it decrements $C[j]$ for valid node j of minimum $W[j]$ in the subtree of i , in a way that it becomes impossible to reach an optimal solution by doing more coefficient decrementing steps. We will call this point in time just before this incorrect decrementing of $C[j]$ “the splitpoint”.

Take an optimal solution which matches the greedy up to the splitpoint, but may have done other decrement steps later. If the optimal solution did a decrement of that same $C[j]$ at any time after the splitpoint, then we could just swap order of decrements and the greedy choice would not have been incorrect.

Thus, we can assume that the optimal solution does not decrement $C[j]$ at all after the splitpoint. Since $S[i] > R$ at the splitpoint and this has to be fixed somehow, the optimal solution has to decrement $C[k]$ for some other node $k \neq j$ in the subtree of i after the splitpoint.

Take one such node k decremented after the splitpoint, such that $LCA(j, k)$ is as deep as possible (farthest from root, closest to j). We can change the optimal solution so that it does one less decrement of $C[k]$, and instead decrements $C[j]$. Note that $W[k] \geq W[j]$ and so if this change is valid we will still have an optimal solution. We will now prove that indeed it is valid.

Nothing outside the subtree of i is changed by this modification. In fact, the only modified $S[x]$ are those for nodes x in the path between nodes j and k (but **excluding their LCA**).

More specifically, an ancestor x of k in the path could in principle now fail the $S[x] \leq R$ check, but that will not be the case because $S[x] \leq R$ was already ensured by BottomUpGreedy before the splitpoint (note that $x \neq i$), thus the extra decrement of $C[k]$ was not needed for this.

On the other hand, an ancestor x of j might now fail the $S[x] \geq L$ check. However, because of the choice of k , the change will not break these conditions: Since j was a valid node at the splitpoint, if it is not possible to do the change it must be because of later decrements affecting node x , but any such decrement would be at a node w having a deeper $LCA(j, w)$ than $LCA(j, k)$, contradicting the choice of k .

Thus in all cases, we reach the conclusion that we can find a solution that agrees with the greedy algorithm for one more step than the splitpoint, contradicting our hypothesis that BottomUpGreedy made a mistake there.

- (Lemma 3) If $W[i] = 0$ for non-leaf node i , then “cutting i from its children” does not change answer. That is, adding the solutions to the resulting trees as if they were each an independent instance of the problem gives an optimal solution to the original.

Proof: $C[i]$ can be set to any value for free. Thus after we choose any solution whatsoever (optimal or not) for the subtrees below i independently, we can set $C[i]$ so that $S[i]$ becomes any desired value in range $[L, R]$ at no cost, and this is always the case regardless of what coefficients were chosen below node i .

Thus after having chosen all coefficients below node i , the remaining problem behaves exactly as if node i were a leaf node itself with $W[i] = 0$, and its solution does not depend at all on the previously chosen coefficients. So, the optimal solution is achieved when an optimal solution is used for each independent subtree below i , and then the remaining weights are chosen ignoring any node below i .

- (Lemma 4) An optimal solution exists with the form of lemma 2, and having either $C[i] = 0$ for every non-leaf node i , or else $S[0] = R$.

Proof: BottomUpGreedy creates such a solution. We can prove it by induction in the number of nodes in the tree. Notice that when processing node 0, if any child j of node 0 has $S[j] = R$, then we have $S[0] \geq R$ and so after fixing it will be exactly $S[0] = R$.

Otherwise, since can inductively assume that for each child j either $S[j] = R$ or the solution was all-zeros, we must now have that all of the non-leaf coefficients are still zero when reaching node 0 in BottomUpGreedy, and so the result also holds.

- (Lemma 5) Let k be the number of leaves in the tree, and m be the minimum $W[i]$ among all the **non-leaf** nodes i . If $kL - R > 0$, then the total cost of an optimal solution is $(kL - R)m + s$, where s is the cost of the optimal solution to a modified input problem in which m is subtracted from $W[i]$ for every **non-leaf** i .

Proof: The “all zero” solution does not work in this case as it would have $S[0] = kL > R$. Thus by lemma 4, we must have $S[0] = R$ instead and so the sum of $C[i]$ over all non-leaf nodes i is exactly $T = R - kL < 0$.

Similar to the reasoning for BottomUpGreedy correctness, we can think of the solution as made up of exactly $-T = kL - R$ “steps” of decrementing some non-leaf coefficient by one, when starting with the $C[i] = 0$ at non-leaves solution.

Let r be a **non-leaf** node of minimum weight, i.e. $W[r] = m$. As a counting trick, we will pretend that we start by setting $C[r]$ to T and the others to zero (note that it might not be a valid solution, but it does not matter as we are

just pretending). Then we already have the $(kL - R)m$ part of the final cost accounted for: it corresponds to this optimistic value where we only use the cheapest node.

Now again thanks to lemma 4, we know that the remaining problem is to choose **exactly** $-T = kL - R$ nodes to “swap” for the optimistic cheapest (note that a node can be chosen multiple times). Thus when choosing a node i for decrement, its actual extra cost we will count as $W[i] - m$, as that is the actual cost that we incur when compared to our reference optimal solution by selecting i instead of r for a decrement.

Crucially, this final step in the proof only works because we know for sure that both before and after the weight adjustment, there are optimal solutions with exactly the same number of “decrements”: exactly $kL - R$, which does not depend on the W values.

Alternative solution for subtasks 6,7

Let $F(C)$ be the sum of $|C[i]| \cdot W[i]$ for all vertices i .

The problem solution is A , the minimum value of $F(C)$, for all possible valid arrays C .

Let $B[u]$ be the number of leaves in subtree of vertex u .

By lemma 4 we know that for subtask 4, A equals to $B[0] \cdot L + \max(0, B[0] \cdot L - R)$

From now on, we might consider more than one weight array W .

Which means the only fixed parts are the given tree and a given query.

Let $F(C, W)$ be $F(C)$ for the given weight array W .

Let $A(W)$ be A for a given weight array W .

Observation: Consider 2 weight arrays W_1 and W_2 , satisfying:

- For any i and j , if $W_1[i] = W_1[j]$, then $W_2[i] = W_2[j]$.
- For any i and j , if $W_1[i] < W_1[j]$, then $W_2[i] \leq W_2[j]$.

Let C be the coefficient array obtained from BottomUpGreedy. Then if we have $F(C, W_1) = A(W_1)$, we must also have $F(C, W_2) = A(W_2)$.

- *Proof:* At each step, if we can choose vertex v for W_1 , then we can also choose v for W_2 .

For a weight array W_1 and an integer T , let $H(W_1, T)$ be another weight array W_2 , where $W_2[i] = 0$ if $W_1[i] \leq T$, and $W_2[i] = 1$ if $W_1[i] > T$.

Observation:

- $A(W) = \sum_{t=0}^{\infty} A(H(W, t))$

- *Proof:* Let C be the coefficient array obtained from BottomUpGreedy. Then for each t , $F(C, H(W, t)) = A(H(W, t))$.

$$\begin{aligned}
\sum_{t=0}^{\infty} A(H(W, t)) &= \sum_{t=0}^{\infty} F(C, H(W, t)) \\
&= \sum_{t=0}^{\infty} \sum_{u=0}^{n-1} [W[u] > t] \cdot C[u] \\
&= \sum_{u=0}^{n-1} C[u] \sum_{t=0}^{\infty} [W[u] > t] \\
&= \sum_{u=0}^{n-1} C[u] \cdot W[u] \\
&= F(C, W) \\
&= A(W).
\end{aligned}$$

Now we can work out $A(W)$ by calculating each $A(H(W, t))$. Notice that this value is 0 for $t \geq \max(W[0 \dots n-1])$.

$H(W, t)$ contains only 0 and 1, which is the same as Subtask 5. The answer is equal to the summation of

$$\text{leaf} \cdot L + \min(0, \text{leaf} \cdot L - R)$$

over all connected components formed by vertices with $W[u] = 1$, where leaf is the number of leaves in this component.

Traverse t from $\max(W[0 \dots n-1])$ to 0. In $H(W, t)$, each vertex u will change from 0 to 1 at point $t = W[u] - 1$. Thus, there will only be $O(n)$ different connected components over all t 's, each will survive in a certain interval of t .

Say, there are $K = O(n)$ connected components, the i -th has $\text{leaf}[i]$ leaves, and its survival time is $\text{life}[i]$. Then the answer is

$$\sum_{i=0}^{K-1} \text{life}[i] \cdot (\text{leaf}[i] \cdot L + \min(0, \text{leaf}[i] \cdot L - R))$$

We can use DSU to work out $\text{leaf}[i]$ and $\text{life}[i]$ in $O(n \log n)$ time.

Subtask 6: 1. In this subtask, $L = 1$ is fixed. 2. Let $\text{ans}[R]$ be the answer for R . 3. For each connected block, its contribution to ans is equivalent to adding a linear function to a prefix and a suffix.

Subtask 7: 1. Rewrite the answer as

$$\left(\sum_{i=0}^{K-1} \text{life}[i] \cdot (\text{leaf}[i] + \min(0, \text{leaf}[i] - R/L)) \right) \cdot L.$$

2. We can apply the same trick as in Subtask 6 for R/L .

IOI 2024 Solutions: Problem Hieroglyphs

Problem Information

Problem Author: Félix Moreno Peñarrubia (Spain)

Problem Preparation: Mohammadreza Maleki, Yiping Liu, Félix M. Peñarrubia

Editorial: Félix Moreno Peñarrubia

Initial Observations

We refer to the values of the elements in the sequences as “characters”, even though the alphabet Σ is not necessarily small. We assume that the size of the alphabet is $|\Sigma| = O(n + m)$ by performing coordinate compression at the beginning if necessary. Note also that we can ignore characters that appear in only one of the sequences. We abbreviate “universal common subsequence” as UCS. Let $\text{occ}_x(A)$ denote the number of occurrences of character x in sequence A . Let $A[i \dots j]$ denote the contiguous subsequence of A from indices i to j inclusive (0-indexed); we will use parenthesis $()$ to denote exclusive ranges at either or both of the endpoints.

Note that, by considering common subsequences of the form $xx \dots x$, we see that if the UCS C of A and B exists, then $\text{occ}_x(C) = \min(\text{occ}_x(A), \text{occ}_x(B))$. So the number of occurrences of each character in the UCS is fixed, and we have to determine the relative order between them.

We say that a character x is A -weak if $\text{occ}_x(A) \leq \text{occ}_x(B)$, and we say x is B -weak if $\text{occ}_x(B) < \text{occ}_x(A)$. Note that we can determine the relative order within A -weak characters and within B -weak characters: the subsequence of A -weak characters of A and the subsequence of B -weak characters of B must be subsequences of C . We denote the subsequence of A -weak characters of A by W_A and the subsequence of B -weak characters of B by W_B .

Subtask 1

Since all characters appear in the UCS, the UCS must have length N , and therefore must be equal to A and also equal to B . Therefore we return A if $A = B$, otherwise return $[-1]$.

Subtask 2

If a character appears only in one of the sequences, we can ignore it. Therefore, we have that $\min(\text{occ}_x(A), \text{occ}_x(B)) = 1$ for all non-ignored characters, and the UCS consists of all distinct characters.

First, we find a candidate for the UCS. We need to merge the sequences W_A and W_B preserving relative order. If x is the first character of W_A and y is the first character of W_B , we can decide which character goes first by checking

whether xy or yx are common subsequences of A and B . If the UCS exists, exactly one of the two sequences should be a common subsequence. We can check it in constant time, and we pop the corresponding character from the beginning of the corresponding sequence (W_A or W_B), add it to the end of the UCS candidate, and repeat until we have merged both sequences.

Once we have the candidate, we have to check if it is indeed a UCS. First, we check if it is a common subsequence. If it is, we can see that the fact that each character appears only once implies that the only way it can fail to be a UCS is if there are two characters x and y so that both xy and yx are common subsequences of A and B . This can only happen if we have the pattern $x \dots y \dots x$ in one sequence and $y \dots x \dots y$ in the other. This can be checked in $O(n \log n)$ using Segment Tree or Fenwick Tree, or in $O(n)$ using stacks.

Subtask 3

When the alphabet is binary, we have a simple characterization of when there is a UCS:

A UCS exists if and only if one sequence is a subsequence of the other or, after removing common prefix and suffix, one of the subsequences only has one of the two characters.

It is clear that if the condition holds there is an UCS, and it is easy to find the UCS. Let us prove that otherwise there is no UCS. Let A and B be sequences without any common prefix or suffix (it is easy to see that after removing common prefix or suffix the property of having or not having UCS is preserved) and such that none is a subsequence of the other. Assume that a UCS exists: we will prove that one of the sequences consists of a repetition of one of the two characters. Note that if a UCS exists and no sequence is a subsequence of the other, then one of the characters must be A -weak and the other must be B -weak, and it cannot happen that there is the same number of occurrences of a character in the two sequences. WLOG assume 0 is A -weak and 1 is B -weak. Note that if A starts with 0 and B with 1, the first character of the UCS can not be uniquely determined, so the UCS does not exist. Same if A ends with 0 and B with 1. Therefore, A starts and ends with 1 and B starts and ends with 0. WLOG assume the UCS starts with 0 (the other case is analogous). Suppose the UCS also contains an 1. Then, since there are strictly more 0s in B than in the UCS, either there are more 0s to the left of the last 1 in B than 0s to the left of the last 1 in the UCS, or there are more 0s to the right of the first 1 in B than 0s to the right of the first 1 in the UCS. We then have that a sequence of the form $0 \dots 01$ or $10 \dots 0$ is a common subsequence of A and B that is not a subsequence of the UCS. Therefore, the UCS can not contain 1s, and therefore B only contains 0s, as desired.

Subtask 4

In this subtask, we have to find a *candidate* for the UCS.

We will merge the sequences W_A and W_B like we did to find a candidate for Subtask 2, but this time we need to handle multiple appearances of characters. If x and y are the characters from W_A and W_B respectively that we are currently comparing, instead of testing whether the sequences xy and yx are common subsequences of A and B , we have to check the sequences $x \dots xy \dots y$ and $y \dots yx \dots x$, where the number of x in the first sequence is 1 plus the number of x s that have already been added to the candidate, and the number of y s is the number of remaining y that have not been added yet to the candidate (and respectively for the second subsequence). We can make the checks in constant time if we precompute, for each character, a list of its positions in each of the subsequences.

Subtask 5

Assuming that we have a candidate for the UCS, we want to verify it in $O(nm)$ time. Note that we can obtain a candidate in $O(nm)$ time by computing the LCS, so it is not necessary to solve the previous subtask to solve this one.

We use dynamic programming. Let $DP[i][j]$ be equal to the minimum k such that all common subsequences of $A[0 \dots i]$ and $B[0 \dots j]$ are subsequences of $C[0 \dots k]$. $DP[i+1][j+1]$ is equal to

- $\max(DP[i+1][j], DP[i][j+1])$ if $A[i] \neq B[j]$,
- $\max(DP[i+1][j], DP[i][j+1], \text{nextocc}(C, A[i], DP[i][j]))$ if $A[i] = B[j]$,

where $\text{nextocc}(C, x, k)$ is the next occurrence of character x in C after index k . We can precompute $\text{nextocc}(C, x, k)$ for all characters x that appear in both sequences and all indices k in $O(nm)$ time (after coordinate compression), so that the transitions can be computed in constant time and DP can be computed in $O(nm)$ total time. If for some combination of indices i, j with $A[i] = B[j]$, $\text{nextocc}(C, A[i], DP[i][j])$ does not exist, then C is not a valid UCS, since there is a common subsequence of $A[0 \dots i]$ and $B[0 \dots j]$ which is not a common subsequence of C .

Full Solution

Following Subtask 4, we already found a candidate C for the UCS. In particular, we assume C satisfies the following:

- C has W_A and W_B as subsequences.
- C is a common subsequence of A and B .

Now we will check whether there exists a common subsequence of A and B that is not a common subsequence of C in $O(n \log n + m \log m)$ time.

Let us define some notation. Given a sequence S and a subsequence T , $I_L^{S,T}$ is the smallest index so that T is a subsequence of $S[0 \dots I_L^{S,T}]$ and $I_R^{S,T}$ is the largest index so that T is a subsequence of $S[I_R^{S,T} \dots |S|]$. Informally, $I_L^{S,T}$ is the last index of subsequence T in S when “going from left to right” and $I_R^{S,T}$ is the first index of subsequence T in S when “going from right to left”. Also, if C is a UCS candidate as above, let $w_L(i) = I_L^{A;C[0 \dots i]}$ if $C[i]$ is an A -weak character, and $w_L(i) = I_L^{B;C[0 \dots i]}$ if $C[i]$ is a B -weak character (similarly define $w_R(i)$).

The key observation lies in the following lemma:

Lemma: Let C be a UCS candidate as above, and suppose that there exists a common subsequence of A and B that is not a subsequence of C . Then, there exists a common subsequence S of A and B which is not a subsequence of C that can be expressed as $S = S_1 + S_2$ satisfying one of the following:

- S_1 ends with an A -weak character, S_2 begins with a B -weak character, $I_L^{A,S_1} = w_L(I_L^{C,S_1})$ and $I_R^{B,S_2} = w_R(I_R^{C,S_2})$.
- S_1 ends with a B -weak character, S_2 begins with an A -weak character, $I_L^{B,S_1} = w_L(I_L^{C,S_1})$ and $I_R^{A,S_2} = w_R(I_R^{C,S_2})$.

Proof: Take S to be a common subsequence of A and B and not a subsequence of C with minimum number of alternations between A -weak characters and B -weak characters. WLOG suppose that S ends with an A -weak character (the other case is analogous). Let S_2 be the suffix of A -weak characters of S , and let S_1 be the remaining characters.

We now prove that this is the desired decomposition. We have that $I_R^{A,S_2} = w_R(I_R^{C,S_2})$ since S_2 consists only of A -weak characters. We have that $I_L^{B,S_1} \leq w_L(I_L^{C,S_1})$ since S_1 is a subsequence of $C[0 \dots I_L^{C,S_1}]$ and $C[0 \dots I_L^{C,S_1}]$ is a subsequence of $B[0 \dots w_L(I_L^{C,S_1})]$. Assume that $I_L^{B,S_1} < w_L(I_L^{C,S_1})$ and consider the sequence S' equal to S_1 concatenated with $C[I_L^{C,S_1}]$ repeated as many times as the character appears in $C[I_L^{C,S_1} \dots |C|]$. By construction, S' is a common subsequence of A and B (it is a subsequence of A because of the inequality $I_L^{A,S_1} < I_R^{A,S_2} = w_R(I_R^{C,S_2}) < w_R(I_L^{C,S_1})$ which is not a subsequence of C , and it has fewer alternations than S , contradiction. Therefore, $I_L^{B,S_1} = w_L(I_L^{C,S_1})$, as desired. $\square$

Now, we use the lemma to perform the check efficiently.

Given two sequences S and T , let $V_L^{S;T}$ be an array of size $|T|$ for which $V_L^{S;T}[i]$ is equal to the lowest index so that there is a subsequence S' of $S[0 \dots V_L^{S;T}[i]]$ for which $I_L^{T;S'} = i$. Similarly define $V_R^{S;T}[i]$ as the highest index so that there is a subsequence S' of $S[V_R^{S;T}[i] \dots |S|]$ for which $I_R^{T;S'} = i$. $V_L^{S;T}$ and $V_R^{S;T}$ can be computed in time $O(|T| \log |T| + |S|)$ using range minimum queries or binary search in cumulative minimum stack.

Compute the vectors $V_L^{A;C}$, $V_R^{A;C}$, $V_L^{B;C}$, $V_R^{B;C}$. By the lemma, if C is not a

UCS, then there exist indices $0 \leq i < j < |C|$ so that either:

- $C[i]$ is A -weak, $C[j]$ is B -weak, $V_L^{A;C}[j] < w_R(i)$ and $V_R^{B;C}[i] > w_L(j)$.
- $C[i]$ is B -weak, $C[j]$ is A -weak, $V_L^{B;C}[j] < w_R(i)$ and $V_R^{A;C}[i] > w_L(j)$.

These conditions can be checked in $O(n \log n)$ time using Segment Tree or Fenwick Tree, or $O(n)$ time using stacks.

IOI 2024 Solutions: Problem Mosaic

Problem Information

Problem Author: Prabowo Djonatan (Indonesia)

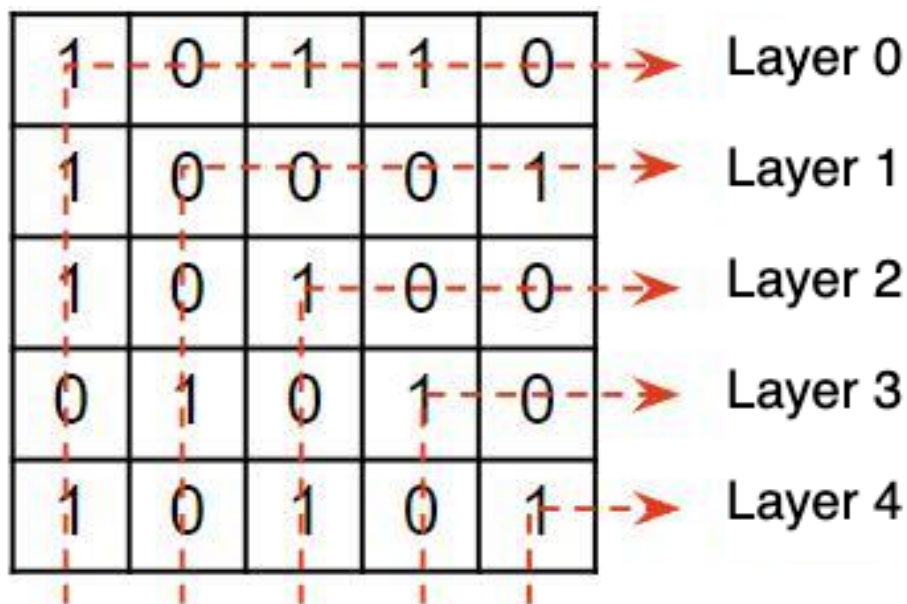
Problem Preparation: Arshia Dadras, Hazem Issa

Editorial: Isaac Moris

Solution

The following is the full solution. The solution for each subtask is on a section after the full solution.

Define layer i as all cells with row x and column y such that $\min(x, y) = i$. After flattening, each layer can be regarded as a sequence of binary numbers.



Observations

The properties of each layer are as follows.

1. **Layer 0:** This is the input layer.
2. **Layer 1:** Contains no two consecutive 1s.
3. **Layer 2:** Contains no two consecutive 1s and no three consecutive 0s.
4. **Layer 3 and beyond:** These layers are derived from Layer 2 by **shrinking**, meaning $A[i][j] = A[i-1][j-1]$.

The repetition of layer 3 and so on can be proven using induction.

Proof for the Main Diagonal

Case 1: $A[i-1][i-1] = 1$

- If $A[i-1][i-1] = 1$, it follows that both $A[i-1][i]$ and $A[i][i-1]$ must be 0 (as per property 3). Thus, we conclude that $A[i][j] = 1$.

Case 2: $A[i-1][i-1] = 0$

- If $A[i-1][i-1] = 0$, then at least one of $A[i-1][i]$ or $A[i][i-1]$ must be 1 (as per property 3). Therefore, we conclude that $A[i][j] = 0$.

Extension to Other Diagonals

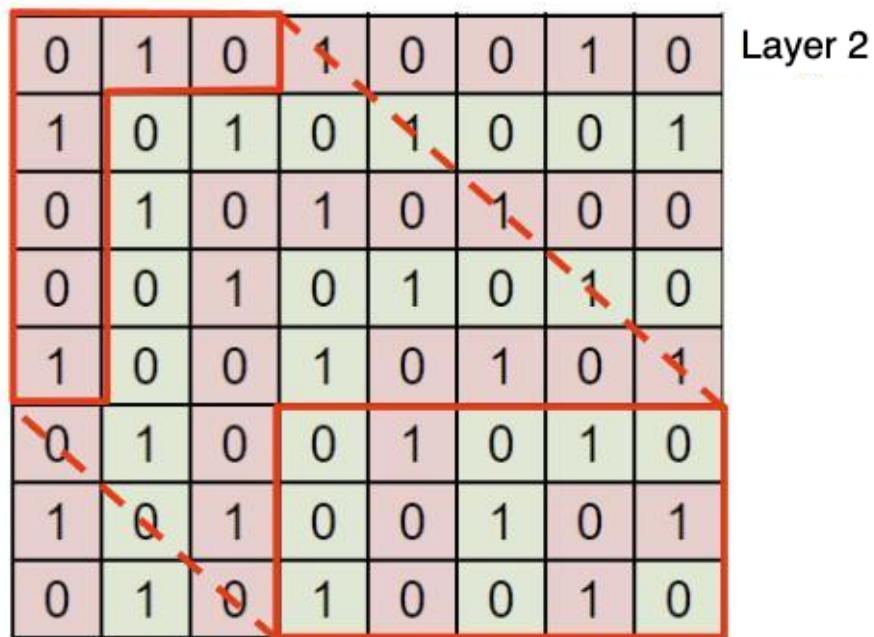
- The same reasoning can be applied to other diagonals. We can begin with the diagonals adjacent to the main diagonal and extend the proof accordingly.
- Notice that $A[i][j-1]$ has the same value of the cell preceding $A[i-1][j-1]$, while $A[i-1][j]$ has the same value of the cell following $A[i-1][j-1]$ in the layer defined by $\min(i-1, j-1)$.

From the observations above, we can start answering the queries.

To answer the query from a scenario, if the subgrid is composed of Layer 0 or Layer 1, then we will count the black cells on those layers separately (this can be done using prefix sum). Hereinafter, we assume subgrids are only composed of Layer 2 or above.

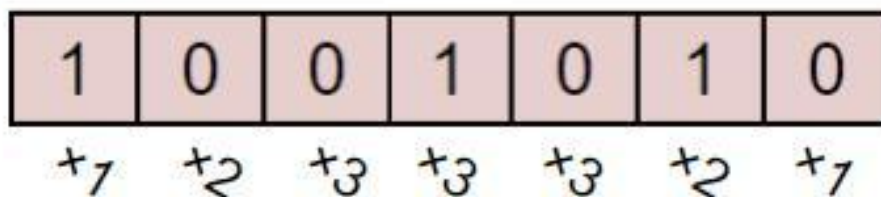
First, we find the projection of the subgrid onto Layer 2. This projection is a continuous subsequence on Layer 2.

As an example,



From the projection, we count the contribution of each element corresponding to the diagonal thickness of the original subgrid.

Notice that the contribution starts with an increasing arithmetic sequence, then it becomes a constant, and finally ends with a decreasing arithmetic sequence.



All of these can also be computed using prefix-sum.

Time complexity: $O(N + Q)$

Subtask 1 and 2

- $N, Q \leq 200$
- In this subtask, we explicitly simulate the resulting subgrid and count the number of black tiles.

Time complexity: $O(QN^2)$

Subtask 3

- $T[k] = B[k] = 0$
- In this subtask, a subgrid only occupies the first row. To solve this subtask, you need to use prefix sum to count the black tiles.

Time complexity: $O(N + Q)$

Subtask 4

- $N \leq 5000$
- In this subtask, the grid is filled explicitly. Then, for each query, the answer is calculated using a 2D prefix sum.

Time complexity: $O(N^2 + Q)$

Subtask 5

- $X[i] = Y[i] = 0$
- In this subtask, layer 0 is completely filled with zeros, while layer 1 and the subsequent layers form a checkerboard pattern. To answer a query, we first exclude layer 0 from the subgrid and calculate the area of the subgrid, dividing the result by two.
- For odd areas, be careful—the answer is given by: $\frac{\text{area} + 1}{2} - (T[k] + B[k]) \bmod 2$

Time complexity: $O(Q)$

Subtask 6

- $T[k] = B[k]$ and $L[k] = R[k]$
- In this subtask, the subgrid consists of a single cell, and its value can be determined by projecting it onto layer 2.

Time complexity: $O(N + Q)$

Subtask 7

- $T[k] = B[k]$
- This subtask requires almost all the observations. However, the contribution of each of the projected elements is 1, so the prefix sum that is needed is much simpler.

Time complexity: $O(N + Q)$

IOI 2024 Solutions: Problem Sphinx

Problem Information

Problem Author: Joshua Lau (Australia)

Problem Preparation: Bartosz Kostka, Mohamed Bakry

Editorial: Bartosz Kostka

Problem Statement

Given a graph with N vertices and M edges, where each vertex has a colour, our goal is to determine the colours by conducting recolouring experiments. A recolouring experiment alters the colours of some vertices and returns the number of monochromatic components in the resulting graph.

Identifying Monochromatic Components

We initially focus on identifying the monochromatic components within the graph without explicitly determining their actual colours (which will give us 50 points).

We construct the monochromatic spanning forest of the graph, where every tree will be a monochromatic component. This can be done inductively. For each new vertex, we perform a binary search to find the appropriate monochromatic component it should be added to. To identify the right component for this new vertex, we temporarily colour all unprocessed vertices to colour N . Now, we should verify whether the new vertex belongs to any already processed component. Then, we can use binary search on the already determined components. We can narrow the binary search by colouring all the components not in the search range to colour N .

Colour Discovery

We can now concentrate on discovering the colours within the pre-determined monochromatic components.

Let us initially examine how to identify colours in the case of a path graph (subtask 3). For a given colour X , we can determine the number of vertices with even and odd indices with this colour separately. Let us first focus on the even vertices. We can modify all the odd vertices to colour N while leaving all the even vertices unchanged. This will give us N monochromatic components. Now let us notice that if we instead assign colour X to each odd vertex, then each even vertex with colour X will decrease the number of monochromatic components by 2 (or 1 if the vertex is at the end of the path), thereby revealing the number of even vertices having colour X . We can replicate this process to determine the count of odd vertices of colour X .

Moreover, we can extend this method to find out the number of vertices with colour X within a specific range by colouring the prefix and suffix outside this range with colour N . Consequently, we can use a binary search to locate the positions of vertices with colour X (first we find the vertex with the smallest index, then find the second smallest, and so on).

Full Solution

Now, let us combine these two approaches.

In the first step, we discovered all monochromatic components, so we can collapse them into single vertices. Then we can build a spanning tree on the collapsed graph, and transform the problem into one where we have two independent sets A and B , with vertices of odd / even depth in this spanning tree. By setting all vertices in A to colour X while leaving all the vertices in B unchanged, whenever a vertex in B also has colour X , it decreases the number of monochromatic components.

Query Count

Finding monochromatic components requires 1 query for each new vertex (to find out if it belongs to any of the already processed monochromatic component) and $\log_2(n)$ queries to identify each new edge of the spanning forest, resulting in a total of $n + (n - 1) \log_2(n)$ queries, since there are at most $n - 1$ edges in the spanning forest.

Determining the colours requires 2 queries per colour (one for A and one for B) and $\log_2(n)$ queries to identify each colour within the collapsed graph.

Note that every edge we find in the colour-connected spanning forest to form the collapsed graph removes a vertex from the collapsed graph, hence the total number of queries is at most $3n + n \log_2(n)$, which for $n = 250$ is less than 2,750, required by the problem.